

AEDS-1 aula de Revisão 1

Não sei o que colocar de subtítulo

Comandos Básicos em C

Comandos Básicos:

Dependências	Comando	Sintaxe	Uso
<stdio.h>	printf()	Ex: printf(“seu texto ...”, var1, var2, ...)	Usado para imprimir algo na tela.
<stdio.h>	scanf()	Ex: scanf(“ %d %d ...”, var1, var2, ...)	Usado para capturar uma entrada digitada pelo usuário
nenhuma	#include	#include	Usado para adicionar bibliotecas
nenhuma	main()	int main(){ ... }	Usado para criar a função principal

Printf():

Sintaxe: `printf("Texto...");`

Dependências: `<stdio.h>`

O `printf()` é uma função usada para imprimir textos na saída padrão (`stdout`), que normalmente é o terminal. Ele imprime o conteúdo entre as aspas exatamente como foi digitado, mantendo caracteres de espaço e tabulação (`tab`). Contudo, há certas sequências de caracteres que possuem um comportamento diferente no `printf()`, e não são impressos exatamente como são digitados entre as aspas. Iremos explorar esses casos a seguir.

Printf():

Imprimindo valores de variáveis:

Quando desejamos imprimir o valor de variáveis, precisamos usar um marcador de posição dentro do texto entre aspas do printf(). Dessa forma a sintaxe do printf() muda um pouco de formato:

Sintaxe: `printf("Texto... %d %f...", var1, var2...);`

Marcadores de posição de variáveis sempre são um caractere de ' % ' seguido de uma letra a depender do tipo de dado da variável (no nosso exemplo: %d e %f). Os marcadores de posição das variáveis servem para 'guardar o local' dentro do texto entre aspas de onde ficará o valor das variáveis, de forma que serão substituídos pelos respectivos valores das variáveis quando impressos na tela. As respectivas variáveis para cada marcador de posição são escritas dentro do printf(), mas após o texto entre aspas e são separadas entre vírgulas.

Printf():

Imprimindo uma quebra de linha (“\n”):

Para o computador, toda quebra de linha é interpretada como um caractere especial de quebra de linha, o “\n” (apesar de serem dois caracteres, eles representam apenas um). Dessa forma, um texto diagramado da seguinte forma:

“tinha uma pedra no meio do caminho
tinha uma pedra
no meio do caminho tinha uma pedra.”

Na memória do computador estará assim:

“tinha uma pedra no meio do caminho\n¹tinha uma pedra\n²no meio do caminho tinha uma
pedra.”

Printf():

Imprimindo uma quebra de linha (“\n”):

Dessa forma comumente colocamos um caractere \n no fim do printf(), para indicar o fim de uma linha.

printf(“Texto\n”);

Assim, futuras execuções do printf(), não irão escrever coisas em uma mesma linha e possivelmente causar confusão. **Obs: o printf() naturalmente não pula uma linha no fim do texto escrito!** Isso significa que se rodarmos os seguintes comandos em sequência:

printf(“Texto1”); printf(“Texto2”) A saída será:

“Texto1Texto2”.

Portanto para diagramar corretamente precisaríamos escrever printf(“Texto1\n”);

printf(“Texto2\n”); Agora sim a saída será:

“Texto1

Texto2

“

Printf():

Imprimindo o sinal de %:

Já vimos que o sinal de % é um sinal usado para indicar um marcador de posição quando acompanhado de uma letra. Ainda assim, se escrevermos o sinal de % sozinho, ele não será impresso pelo printf(). Isso se dá porque o printf() estará esperando que você insira uma letra após o sinal de % porque ele está esperando um marcador de posição. Caso queira imprimir exatamente o sinal de % é preciso digitar 2 sinais de porcentagem seguidos: ‘%%’. Dessa forma o printf() irá entender corretamente que você deseja imprimir o sinal de % e irá imprimir apenas um na tela.

printf(“%%”);

Saída: %

Printf():

Caracteres ç, ã, á, à, â, e outros acentos

Se você tentar imprimir algum caractere com acento ou um ‘ç’ por exemplo, usando o printf() irá perceber que vários caracteres estranhos serão impressos na tela. Isso ocorre por alguns motivos internos de memória do programa em C, mas como a nossa primeira análise acerca desse assunto é apenas superficial, cabe dizer que não é possível imprimir caracteres de acento (e alguns outros caracteres especiais também) usando o printf() em C sem lançar mão de uma biblioteca específica para tal.

Scanf():

Sintaxe: `scanf("%d %f ...", &var1, &var2, ...);`

Dependências: `<stdio.h>`

O `scanf()` é uma função usada para capturar valores inseridos pelo usuário na entrada padrão (`stdin`) que normalmente é o terminal (valores digitados pelo teclado do usuário). Ele captura tipos de dados inseridos pelo usuário de acordo com os especificados pelos marcadores de posição dentro das aspas, e armazena-os nas variáveis que seguem ao lado das aspas em ordem. Para que o `scanf()` consiga armazenar corretamente os valores nas variáveis, é necessário o uso do caractere `'&'` antes do nome de cada variável.

Scanf():

Observações:

O uso do caractere ‘&’ antes do nome das variáveis têm um significado específico, porém complexo, e que será abordado mais adiante no curso. Por enquanto, a melhor abordagem é realmente memorizar que no scanf() o seu uso é necessário.

É aconselhável escrever um caractere de espaço antes dos marcadores de posição e dentro das aspas do scanf() porque isso evita alguns comportamentos inesperados que ocorrem em certos casos usando variáveis do tipo char.

```
scanf(" %d %f ...", &var1, &var2, ...);
```

Tais casos também serão abordados com mais ênfase posteriormente no curso.

include:

Sintaxe: **#include <nome da biblioteca>;**

Dependências: **nenhuma**

O comando #include é o utilizado para incluir bibliotecas no seu código. Bibliotecas serão explicadas de uma forma mais profunda mais posteriormente no curso, porém, por enquanto podemos entendê-las como um conjunto de códigos que possibilitam que outros comandos específicos sejam executados. Sendo assim, se um comando tem uma determinada biblioteca como dependência, essa biblioteca precisa ser adicionada no seu código para que aquele comando possa ser executado. A inclusão dessas bibliotecas no código é feita usando o comando #include seguido do nome da biblioteca entre '<' e '>' .

main():

Sintaxe: `int main(){ ... }`

Dependências: **nenhuma**

Não se caracterizando especificamente como um comando em si, mas sim uma função (ainda iremos trabalhar o conceito de “função”), resumidamente pode ser tratada como a entidade principal do seu programa. Todos os comandos que você queira que sejam executados devem estar dentro das chaves (‘{’ e ‘}’) da main. Há certos comandos que devem ser escritos fora da main() e inclusive podem ser executados antes dela, contudo, por enquanto iremos tomar a máxima de que qualquer coisa que esteja fora da main() não será executado pelo programa.

OBS: a main() sempre termina com o comando `return 0` antes da sua chave final, não se esqueça de incluí-lo.

Tipos de dados e declaração de Variáveis

Variáveis

A todo momento no código será preciso lidar com dados, sejam eles inseridos pelo usuário ou sejam eles produzidos pelo próprio programa. Fazemos isso usando as variáveis. Elas representam um espaço na memória do programa reservado para o armazenamento de um determinado dado, que pode ser numérico ou literal. A partir das variáveis, é possível manipular os dados conforme for conveniente para criarmos os mais variados programas, o que as tornam fundamentais para o código. Quando uma variável é criada, ela é especificada para comportar um determinado tipo de dado, não suportando formatos de dados diferentes ao que lhe foi designada a armazenar. Para isso, temos os tipos de dados

Tipos de Dados

A linguagem C possui nativamente 5 tipos de dados:

char

int

float

double (long float)

void*

Os tipos de dados determinam o conteúdo que as variáveis podem armazenar, assim como características de como elas lidam com esses dados.

*O tipo de dado void é um tipo de dado especial que não pode ser usado para criar variáveis, ele será tratado mais a frente detalhadamente e está aqui apenas a título de curiosidade.

char

Tamanho em bytes: 1 byte Marcador de posição: %c

Faixa de valores: -128 a 127

O tipo de dado char é designado a armazenar caracteres. A variedade de caracteres que um char pode armazenar é limitada, e será estudada em profundidade mais adiante. Por enquanto, é importante saber que qualquer caractere literal (seja a letra maiúscula ou minúscula), numérico e símbolos simples como '!', '? ' ou ' * ', ele é capaz de armazenar. Outro aspecto que será estudado profundamente mais adiante é a possibilidade de realizar operações aritméticas com um um char, por agora, é suficiente dizer que não é possível.

int

Tamanho em bytes: 4 bytes Marcador de posição: %d ou %i

Faixa de valores: -2.147.483.648 a 2.147.483.647

O tipo de dado int é designado a armazenar números inteiros. Ele pode armazenar qualquer número inteiro dentro da sua faixa de valores e (assim como qualquer outro tipo de dado numérico) suporta operações aritméticas com seus valores. Quando uma divisão é realizada usando tipos de dado int, ela será tratada como uma divisão inteira, ou seja, caso o quociente resulte em algum número decimal, a parte decimal desse número será descartada sem arredondamentos. **Por enquanto, é o único tipo de dado que suporta o operador mod (%), já que ele significa justamente o resto da divisão inteira (o operador mod será explicado em detalhes mais adiante).**

float

Tamanho em bytes: 4 bytes Marcador de posição: %f

Faixa de valores: $\sim \pm 3.4 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$ (6-7 dígitos de precisão)

O tipo de dado float é designado a armazenar números reais (com ou sem casas decimais). Ele pode armazenar qualquer número real dentro da sua faixa de valores com uma aproximação para 6 casas decimais e (assim como qualquer outro tipo de dado numérico) suporta operações aritméticas com seus valores. Quando uma divisão é realizada usando tipos de dado float, ela será tratada como uma divisão exata, ou seja, o resultado do quociente será exato, com uma aproximação para 6 casas decimais. O tipo de dado float não suporta o operador mod (%) e teremos comportamento inesperado no código se tentarmos usá-lo com variáveis float.

double

Tamanho em bytes: 8 bytes Marcador de posição: %lf

Faixa de valores: $\sim \pm 1.7 \times 10^{-308}$ a $\pm 1.7 \times 10^{308}$ (15-16 dígitos de precisão)

O tipo de dado double também é designado a armazenar números reais porém com mais casas decimais (maior precisão). Ele pode armazenar qualquer número real dentro da sua faixa de valores com uma aproximação para 15 casas decimais e (assim como qualquer outro tipo de dado numérico) suporta operações aritméticas com seus valores. Uma divisão entre tipos de dados double funciona exatamente igual a uma divisão entre float, porém com uma maior precisão no resultado. Vale lembrar, que o double também é capaz de armazenar números maiores que o float. **O tipo de dado double também não suporta o operador mod (%) e teremos comportamento inesperado no código se tentarmos usá-lo com variáveis double.**

bool

Tamanho: 1 byte

Faixa de valores: 0 ou 1

Dependências: <stdbool.h>

Apesar de o tipo de dado booleano não ser nativo em C, podemos usá-lo ao incluir a biblioteca `stdbool.h`. O tipo de dado `bool` armazena um valor booleano, ou seja, de verdadeiro ou falso, (1 ou 0) não podendo assumir nenhum outro valor. Esse tipo de dado é muito usado em operações lógicas e verificações de verdadeiro ou falso em condicionais ou loops. Não é correto realizar operações aritméticas com o `bool`, assim como não é possível ler um valor booleano da entrada diretamente usando `scanf()`. Podemos, contudo, imprimir um valor booleano usando o `printf()` e o marcador de posição de inteiro (`%d`), e, quando incluirmos a biblioteca `stdbool.h` passamos a poder usar "true" ou "false" para expressões verdadeiras e falsas respectivamente.

Declaração de Variáveis

Para criarmos uma variável no código, basta escrevermos o tipo de dado da variável que será criada, e então escrevermos o nome que queremos dar à ela:

Sintaxe: **tipo de dado** **nome da variável** ;

Ex: **int variavel** ;

No exemplo, criamos uma variável com o nome de “variavel” e que é do tipo inteiro, ou seja, irá armazenar dados numéricos inteiros. Podemos também declarar uma variável já com um valor armazenado nela:

Ex: **int variavel = 2** ;

Fazemos isso usando um sinal de = após o nome da variável. Assim, no momento de sua criação, a variável “variavel” que é do tipo int, já terá o valor 2 armazenado.

Declaração de Variáveis

Observações:

É recomendável declarar uma variável já com um valor inicial para evitar o erro de operar variáveis vazias. Normalmente o valor inicial dado a variáveis de dados numéricos é zero, mas pode ser um outro arbitrário que faça sentido no seu programa.

Variáveis do mesmo tipo de dado podem ser declaradas em sequência, separadas por vírgula e terminadas com um ponto e vírgula no fim da sentença:

Ex: `int var1 = 0, var2 = 0, var3 = 1;`

No exemplo, todas as variáveis são declaradas corretamente, cada uma com o seu valor inicial, e todas do tipo int.

Nomes de variáveis

Há algumas regras que precisam ser seguidas com relação ao nome de uma variável:

- 1) Apenas os caracteres de letras, números e o caractere `\_` são permitidos
- 2) O nome não pode começar com um caractere de número
- 3) Não pode haver nenhum tipo de espaço no nome da variável
- 4) O nome não pode ser exatamente igual ao de uma palavra reservada em C

Palavras reservadas em C são aquelas que designam algum comando em C como por exemplo: `int` (usado para criar variáveis `int`), `return` (usado no fim de funções), etc...

OBS: Os caracteres de letras maiúsculas são diferenciados dos caracteres de letras minúsculas. Ex: as variáveis : `"Nome"` e `"nome"` são diferentes, assim como `"NOME"` e `"nome"`.

Operadores

Operadores

Para realizarmos operações em C, sejam elas aritméticas ou lógicas, usamos os operadores. Eles são componentes de uma expressão que será resolvida pelo programa a fim de produzir efeitos variados no código. Expressões no código, nada mais são do que uma linha de código contendo números, operadores e/ou variáveis aonde uma ou mais operações serão realizadas a fim de produzir um efeito específico.

Ex: variavel = 2 + 3;

No exemplo, usamos o operador de adição, os números 2 e 3, e a variável “variavel”. Essa expressão, agrega na variável “variavel” o valor de 2 + 3, portanto, ao final dessa linha de código ser lida, “variavel” terá o valor 5.

Para entendermos melhor o que podemos fazer dentro de uma expressão, vamos analisar como funcionam cada um dos operadores em C:

Operador de Atribuição (=).

Da mesma forma que podemos atribuir um valor inicial à uma determinada variável no momento de sua declaração, podemos atribuir um valor à uma variável já criada em qualquer parte do código. Fazemos isso usando o operador de atribuição(=). Ao atribuirmos um valor à uma variável, qualquer valor que ela possuía anteriormente é descartado e substituído pelo novo atribuído;

Ex: **var = 3;**

Esse comando substituí o antigo valor que a variável “var” possuía pelo valor 3. Podemos também, atribuir o resultado de uma expressão à uma variável:

Ex: **var = (expressão);**

Nesse caso, a expressão será calculada e então atribuída à variável “var”

Operadores Aritméticos

+ - * /

Assim como na matemática, os operadores (+ - * /) representam respectivamente a adição, a subtração, a multiplicação e a divisão. Podemos montar expressões como na matemática, e elas serão executadas da esquerda para a direita, respeitando a ordem de prioridade entre as operações.

Ex: 2 + 3 / 3 * 2 - 1

A seguinte operação irá resultar em 3, já que primeiramente a divisão 3 / 3 é feita, e então a multiplicação 1 * 2 é feita, para por fim ser feito 2 + 2 e então subtrair 1.

Cuidado! Não podemos montar equações usando os operadores

Ex: 2 + 3 / 3 * 2 - 1 = x - 2

O exemplo mostra uma sentença errada porque o operador = no código significa o operador de atribuição e portanto o código não irá calcular o valor de x.

Operadores Aritméticos

Operador de mod (%)

Além dos operadores que representam as 4 operações fundamentais da matemática, ainda temos um operador aritmético que é o operador de módulo, ou mod. O símbolo dele é o símbolo de porcentagem (%) e ele representa o resto da divisão inteira entre dois números, e, por esse motivo também, só pode ser usado com tipos de dados numéricos inteiros.

Ex: `var = 5 % 3`

No exemplo, ao final dessa expressão, a variável “var” irá armazenar o valor de 2, já que 5 dividido por 3 é igual a 1 e possui resto 2.

Operadores Aritméticos

Operadores reduzidos

Suponhamos que temos uma variável “int var” que já está armazenando o valor de 5, e queremos somar mais 5 à essa variável para que ela valha 10. Observe então a seguinte sentença:

Ex: var + 5;

Observe que ao realizarmos a seguinte expressão, pegaremos o valor de “var” que é 5 e iremos somar 5 à ele obtendo 10 como resultado, porém esse resultado não está sendo guardado em nenhum lugar, de forma que ele se perde no final dessa expressão, e a variável “var” permanece contendo o valor de 5.

Se quiséssemos que algo armazenasse o valor do resultado 10 da expressão, teríamos que usar o operador de associação com uma variável, como no exemplo a seguir:

Ex: int x = var + 5;

Agora a variável “x” está armazenando o valor 10.

Operadores Aritméticos

Operadores reduzidos

Contudo o que queríamos desde o começo era que a própria variável “var” armazenasse o valor 10 ao final da expressão, portanto podemos usá-la em vez da variável “x” para armazenar o resultado da expressão:

Ex: `var = var + 5;`

Como a expressão que resulta no valor que a variável irá armazenar é resolvida primeiramente, então temos que `var + 5` resulta em 10, já que o valor de `var` antes da associação era 10, e então associamos esse valor 10 na própria variável `var`.

Essa seria a forma apropriada de somar o valor de 5 ao valor já contido em `var`.

Operadores Aritméticos

Operadores reduzidos

`+=` , `-=` , `*=` , `/=` e `%=`

Contudo, C nos oferece uma alternativa mais prática do que escrever duas vezes o nome da variável à qual queremos somar um valor para armazenar o valor da soma na própria variável, e a alternativa são justamente os Operadores Reduzidos. O operador `+=` realiza exatamente o que queríamos no nosso exemplo, ele soma um valor arbitrário ao valor original de uma determinada variável, e armazena o resultado na própria variável

Ex: `var += 5;`

Ao final dessa expressão, `var` irá conter o valor de 10, já que o operador somou 5 ao valor antigo da variável e armazenou o resultado na própria variável `var`.

Operadores Aritméticos

Operadores reduzidos

`+=` , `-=` , `*=` , `/=` e `%=`

De forma análoga, temos um operador reduzido para cada um dos operadores aritméticos de C. Todos eles seguem a mesma lógica do de adição, porém com sua operação correspondente: irão efetuar a operação da variável pelo valor à direita, e armazenar o resultado na própria variável.

Ex: `var /= 5;`

Ao final dessa expressão, var irá conter o valor de 1, já que primeiramente será efetuado `5 / 5` (valor antigo de var era 5), e então armazenar o resultado na própria variável var.

Operadores Lógicos

< , <= , > , >= , == , != , && , ||

Os operadores aritméticos servem para realizarmos contas e operações matemáticas com os dados do nosso código. Porém, além de operações aritméticas, outras operações importantíssimas, na programação são as operações lógicas, que são aquelas que resultam apenas em verdadeiro, ou falso. Em resumo, uma operação lógica pode ser considerada como uma pergunta de SIM ou NÃO.

Ex: 2 < 5

Perceba que no exemplo temos uma pergunta: “2 é menor do que 5?” Como a resposta é “sim” o valor do resultado dessa operação lógica será de “verdadeiro”

As operações lógicas sempre terão esse valor de realizar a verificação da veracidade de uma sentença, se ela for verdadeira, o resultado tem valor de “verdadeiro”, senão, tem valor de “falso”.

Operadores Lógicos

< , <= , > , >= , == , != , && , ||

Para isso temos em resumo dois tipos de operadores lógicos: aqueles que verificam relações entre valores numéricos, e aqueles que verificam relações entre valores lógicos. Para aqueles que verificam relações entre valores numéricos temos:

$x < y$: x menor que y

$x <= y$: x menor ou igual a y

$x > y$: x maior que y

$x >= y$: maior ou igual a y

$x == y$: x igual a y

$x != y$: x diferente de y

Operadores Lógicos

Observação:

Perceba que se queremos verificar se dois valores de dados são iguais, usamos o operador lógico “==” e não apenas o “=”. Tomemos novamente como exemplo o valor da variável “int var” como sendo 5:

Ex: **var = 5**

Esse exemplo usa o operador de associação (=) e portanto, não verifica se o valor contido em “var” é 5, apenas agrega o valor de 5 em “var”, sobrescrevendo o valor antigo contido em “var”.

Ex: **var == 5**

Esse exemplo usa o operador de verificação de igualdade (==) e, agora sim, verifica se o valor contido em “var” é 5, portanto, essa expressão terá valor de “verdadeiro”.

Operadores Lógicos

&& , ||

Por fim, temos os operadores lógicos que relacionam dois outros valores lógicos, o operador AND (&&) e o operador OR (||). Tais operadores estão diretamente ligados à álgebra booleana e seguem o mesmo padrão da tabela verdade das operações booleanas de AND e OR:

AND (&&):

p	q	$p \wedge q$
V	V	V
V	F	F
F	V	F
F	F	F

OR(||):

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

Operadores Lógicos

Operador NOT (!)

O operador NOT (!) é um operador que atua em apenas um valor ou expressão lógico, alternando o seu valor entre verdadeiro e falso. Se o valor antes era verdadeiro, passa a ser falso, se era falso, passa a ser verdadeiro:

Ex: ! (5 < 6)

No exemplo, a expressão resulta em valor falso, porque a expressão entre parêntesis tem valor positivo e então será invertido para falso devido ao operador NOT.

Operadores

Observação:

Para todos os tipos de expressões é possível usar os parêntesis para especificar quais operações devem ser realizadas primeiramente. As operações dos parêntesis mais internos sempre têm prioridade e, quando não houverem mais prioridade de parêntesis, elas são seguidas na ordem da esquerda para a direita.

Ex: ((5 < 3) || (6 >= 6)) && (4 <= 7)

O resultado dessa expressão será verdadeiro. Primeiramente será efetuado $5 < 3$ que é falso, e $6 \geq 6$ que é verdadeiro. Então será realizado o OR de ambos esses resultados, resultando em verdadeiro. Em seguida será efetuado $4 \leq 7$ que é verdadeiro, e então será calculado o AND desse resultado com o anterior, o que irá gerar Verdadeiro.

OBS: É sempre bom deixar bem claro a ordem das operações nas expressões usando os parêntesis para melhorar a legibilidade do seu código.